

Architektury systemów komputerowych

Lista zadań nr 6

Na zajęcia 8 i 21 kwietnia 2026

Należy być przygotowanym do wyjaśnienia semantyki każdej instrukcji, która pojawia się w treści zadania. W tym celu posłuż się dokumentacją: [x86 and amd64 instruction reference](http://www.felixcloutier.com/x86/)¹. W szczególności trzeba wiedzieć jak dana instrukcja korzysta z rejestru flag EFLAGS tam, gdzie obliczenia zależą od jego wartości.

W trakcie tłumaczenia kodu z assemblera x86-64 do języka C należy trzymać się następujących wytycznych:

- Używaj złożonych wyrażeń minimalizując liczbę zmiennych tymczasowych.
- Nazwy wprowadzonych zmiennych muszą opisywać ich zastosowanie, np. `result` zamiast `rax`.
- Instrukcja `goto` jest zabroniona. Należy używać instrukcji sterowania `if`, `for`, `while` i `switch`.
- Pętle `while` należy przetłumaczyć do pętli `for`, jeśli poprawia to czytelność kodu.

Uwaga! Przedstawienie rozwiązania niestosującego się do powyższych zasad może skutkować negatywnymi konsekwencjami.

Zadanie 1. Poniższy wydruk otrzymano w wyniku deasemblacji rekurencyjnej procedury zadeklarowanej następująco: `long pointless(long n, long *p)`. Zapisz w języku C kod odpowiadający tej procedurze. Następnie opisz zawartość jej rekordu aktywacji (ang. *stack frame*). Wskaż rejestry zapisane przez funkcję wołaną (ang. *callee-saved registers*), zmienne lokalne i miejsce, na które będzie wskazywał adres powrotu. Następnie uzasadnij, że wartość rejestru `%rsp` w wierszu 11 jest podzielna przez 16 – zgodnie z [1, 3.2.2]. Zastanów się czemu autorzy ABI zdecydowali się na taką konwencję.

```
1  pointless:                                11      callq  pointless
2      pushq  %r14                            12      addq   (%rsp), %rax
3      pushq  %rbx                            13      jmp    .L3
4      pushq  %rax                            14 .L1:  xorl   %eax, %eax
5      movq   %rsi, %r14                      15 .L3:  addq   %rax, %rbx
6      movq   %rdi, %rbx                      16      movq   %rbx, (%r14)
7      testq  %rdi, %rdi                      17      addq   $8, %rsp
8      jle    .L1                             18      popq   %rbx
9      leaq   (%rbx,%rbx), %rdi                19      popq   %r14
10     movq   %rsp, %rsi                       20      retq
```

Wskazówka: Ta funkcja nie ma konkretnego zastosowania.

Zadanie 2. Poniżej zamieszczono kod rekurencyjnej procedury o sygnaturze `long puzzle2(long *a, long v, uint64_t s, uint64_t e)`. Przetłumacz tę procedurę na język C, a następnie jednym zdaniem powiedz co ona robi.

```
1  puzzle2:                                11      jg     .L11
2      movq   %rcx, %rax                      12      leaq   1(%rax), %rdx
3      subq   %rdx, %rax                      13      call   puzzle2
4      shrq   %rax                            14 .L10: ret
5      addq   %rdx, %rax                      15 .L11: leaq   -1(%rax), %rcx
6      cmpq   %rdx, %rcx                      16      call   puzzle2
7      jb     .L5                             17      ret
8      movq   (%rdi,%rax,8), %r8              18 .L5:  movq   $-1, %eax
9      cmpq   %rsi, %r8                      19      ret
10     je     .L10
```

Wskazówka: Z reguły procedurę `puzzle2` woła się następująco: `i = puzzle2(a, v, 0, n - 1)`.

¹<http://www.felixcloutier.com/x86/>

Zadanie 3. Zakładamy, że producent procesora nie dostarczył instrukcji skoku (czyli żadnej instrukcji postaci `jmpq`). Rozważmy procedurę `switch_prob` z poprzedniej listy. Podaj metodę zastąpienia instrukcji `jmpq *0x4006f8(,%rsi,8)` ciągiem innych instrukcji. Nie można używać kodu *samomodyfikującego się* (ang. *self-modifying code*), ani dodatkowych rejestrów.

Napisz kod w języku C, który wygeneruje instrukcję *pośredniego wywołania procedury*, na przykład `call *(%rdi,%rsi,8)`, a następnie zaprezentuj go posługując się stroną [godbolt²](https://godbolt.org). Pokaż, że taką instrukcję też da się zastąpić, gdyby brakowało jej w zestawie instrukcji.

Zadanie 4. Poniżej widnieje kod wzajemnie rekurencyjnych procedur M i F typu `long (*) (long)`. Programista, który je napisał, nie pamiętał wszystkich zasad *konwencji wołania procedur*. Wskaż co najmniej dwa różne problemy w poniższym kodzie i napraw je! Następnie przetłumacz kod do języka C.

1 M: pushq %rdi	12 F: testq %rdi, %rdi
2 testq %rdi, %rdi	13 je .L3
3 je .L2	14 movq %rdi, %r12
4 leaq -1(%rdi), %rdi	15 leaq -1(%rdi), %rdi
5 call M	16 call F
6 movq %rax, %rdi	17 movq %rax, %rdi
7 call F	18 call M
8 movq (%rsp), %rdi	19 subq %rax, %r12
9 subq %rax, %rdi	20 movq %r12, %rax
10 .L2: movq %rdi, %rax	21 ret
11 ret	22 .L3: movl \$1, %eax
	23 ret

Zadanie 5. Skompiluj poniższy kod źródłowy kompilatorem `gcc` z opcjami `-Og -fomit-frame-pointer -fno-stack-protector` i podejrzaj wygenerowany kod assemblera, ponownie posługując się stroną [godbolt](https://godbolt.org). Wskaż w kodzie maszynowym instrukcje realizujące przydział tablicy `p` i narysuj rekord aktywacji procedury. Wyjaśnij co robi instrukcja `leave`.

```
1 long aframe(long n, long idx, long *q) {
2     long i;
3     long *p[n];
4     p[n-1] = &i;
5     for (i = 0; i < n; i++)
6         p[i] = q;
7     return *p[idx];
8 }
```

Zadanie 6. Poniżej widnieje kod procedury o sygnaturze `long puzzle6(void)`. Narysuj rekord aktywacji procedury `puzzle6`, podaj jego rozmiar i składowe. Procedura `readlong`, która wczytuje ze standardowego wejścia liczbę całkowitą, została zdefiniowana w innej jednostce translacji. Jaka jest jej sygnatura? Przetłumacz procedurę `puzzle6` na język C i wytłumacz jednym zdaniem co ona robi.

1 puzzle6:	8 cqto
2 subq \$24, %rsp	9 idivq 8(%rsp)
3 movq %rsp, %rdi	10 xorl %eax, %eax
4 call readlong	11 testq %rdx, %rdx
5 leaq 8(%rsp), %rdi	12 sete %al
6 call readlong	13 addq \$24, %rsp
7 movq (%rsp), %rax	14 ret

²<https://godbolt.org>

Zadanie 7 (bonus). Procedurę ze zmienną liczbą parametrów używającą pliku nagłówkowego `stdarg.h`³ skompilowano z opcjami `-Og -mno-sse`. Po jej deasemblacji otrzymano następujący wydruk. Przetłumacz procedurę `puzzle7` na język C i wytłumacz jednym zdaniem co ona robi. Narysuj rekord aktywacji procedury, a następnie podaj jego rozmiar i składowe. Prezentację zacznij od przedstawienia definicji struktury `va_list` na podstawie [1, 3.5.7].

```
1 puzzle7:                                14 .L3:  movq  -64(%rsp), %rdx
2     movq  %rsi, -40(%rsp)                15     leaq  8(%rdx), %rcx
3     movq  %rdx, -32(%rsp)                16     movq  %rcx, -64(%rsp)
4     movq  %rcx, -24(%rsp)                17 .L4:  addq  (%rdx), %rax
5     movq  %r8, -16(%rsp)                18 .L2:  subq  $1, %rdi
6     movq  %r9, -8(%rsp)                  19     js   .L6
7     movl  $8, -72(%rsp)                  20     cmpl  $47, -72(%rsp)
8     leaq  8(%rsp), %rax                   21     ja   .L3
9     movq  %rax, -64(%rsp)                22     movl  -72(%rsp), %edx
10    leaq  -48(%rsp), %rax                 23     addq  -56(%rsp), %rdx
11    movq  %rax, -56(%rsp)                24     addl  $8, -72(%rsp)
12    movl  $0, %eax                       25     jmp  .L4
13    jmp  .L2                             26 .L6:  ret
```

Literatura

- [1] „*System V Application Binary Interface AMD64 Architecture Processor Supplement*”
<https://raw.githubusercontent.com/wiki/hjl-tools/x86-psABI/x86-64-psABI-1.0.pdf>

³<https://en.wikipedia.org/wiki/Stdarg.h>